

JDBC & HQL QUICK SYNTAX SHEET

(Complete Exam Revision Guide)

JDBC Quick Syntax Sheet (Exam Revision)

1. Import JDBC Package

```
import java.sql.*;
```

2. Load JDBC Driver

MySQL JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

3. Create Connection

```
String url = "jdbc:mysql://localhost:3306/studentdb";  
String user = "root";  
String password = "root";  
  
Connection con = DriverManager.getConnection(url, user, password);
```

4. Statement (Normal SQL)

Create Statement Object

```
Statement stmt = con.createStatement();
```

SELECT

```
ResultSet rs = stmt.executeQuery(  
    "SELECT * FROM student"  
);
```

INSERT

```
int rows = stmt.executeUpdate(  
    "INSERT INTO student VALUES(101,'Indra',85)"  
);
```

UPDATE

```
int rows = stmt.executeUpdate(  
    "UPDATE student SET marks=90 WHERE id=101"  
);
```

DELETE

```
int rows = stmt.executeUpdate(  
    "DELETE FROM student WHERE id=101"  
);
```

5. PreparedStatement

INSERT

```
String sql = "INSERT INTO student(id,name,marks) VALUES(?,?,?)";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1,101);
ps.setString(2,"Indra");
ps.setInt(3,85);

ps.executeUpdate();
```

SELECT

```
String sql = "SELECT * FROM student WHERE id=?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1,101);

ResultSet rs = ps.executeQuery();
```

UPDATE

```
String sql = "UPDATE student SET marks=? WHERE id=?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1,95);
ps.setInt(2,101);

ps.executeUpdate();
```

DELETE

```
String sql = "DELETE FROM student WHERE id=?";
```

```
PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1,101);

ps.executeUpdate();
```

6. ResultSet

```
while(rs.next())
{
    System.out.println(rs.getInt("id"));
    System.out.println(rs.getString("name"));
    System.out.println(rs.getInt("marks"));
}
```

7. ResultSetMetaData

```
ResultSetMetaData md = rs.getMetaData();

System.out.println(md.getColumnCount());
System.out.println(md.getColumnName(1));
```

8. Transaction Management

Disable Auto Commit

```
con.setAutoCommit(false);
```

Commit

```
con.commit();
```

Rollback

```
con.rollback();
```

Savepoint

```
Savepoint sp = con.setSavepoint();  
  
con.rollback(sp);
```

9. Batch Processing

```
Statement stmt = con.createStatement();  
  
stmt.addBatch(  
    "INSERT INTO student VALUES(101,'A',80)"  
);  
  
stmt.addBatch(  
    "INSERT INTO student VALUES(102,'B',90)"  
);  
  
stmt.executeBatch();
```

10. Close Resources

```
rs.close();  
stmt.close();  
con.close();
```

JDBC CRUD Summary

Operation	Method
SELECT	executeQuery()
INSERT	executeUpdate()
UPDATE	executeUpdate()
DELETE	executeUpdate()
COMMIT	con.commit()
ROLLBACK	con.rollback()
SAVEPOINT	con.setSavepoint()
BATCH	executeBatch()

JDBC Flow:

Load Driver → Create Connection → Create Statement / PreparedStatement → Execute SQL Query → Process ResultSet → Commit/Rollback (if needed) → Close Connection

HQL (Hibernate Query Language) Quick Syntax Sheet

Note: HQL works on Java Class Names and Object Properties, not on table names and column names.

Suppose:

```
@Entity
public class Student {
    private int id;
    private String name;
    private int marks;
}
```

1. SELECT (Read)

Select All Records

```
String hql = "FROM Student";

Query<Student> q = session.createQuery(hql, Student.class);

List<Student> list = q.list();
```

Select with Condition

```
String hql = "FROM Student WHERE id = :id";

Query<Student> q = session.createQuery(hql, Student.class);

q.setParameter("id", 101);

List<Student> list = q.list();
```

Select Specific Columns

```
String hql = "SELECT name, marks FROM Student";
```

2. INSERT

HQL does not support normal INSERT statements. Use:

```
Student s = new Student();

s.setId(101);
s.setName("Indra");
s.setMarks(85);

session.save(s);

// or

session.persist(s);
```

3. UPDATE

HQL Update Query

```
String hql = "UPDATE Student SET marks = :m WHERE id = :id";

Query q = session.createQuery(hql);

q.setParameter("m", 90);
q.setParameter("id", 101);

q.executeUpdate();
```


4. DELETE

HQL Delete Query

```
String hql = "DELETE FROM Student WHERE id = :id";

Query q = session.createQuery(hql);

q.setParameter("id", 101);

q.executeUpdate();
```

5. ORDER BY

```
String hql = "FROM Student ORDER BY marks DESC";
```

6. Aggregate Functions

```
String hql = "SELECT COUNT(*) FROM Student";

String hql = "SELECT MAX(marks) FROM Student";

String hql = "SELECT MIN(marks) FROM Student";

String hql = "SELECT AVG(marks) FROM Student";

String hql = "SELECT SUM(marks) FROM Student";
```

7. GROUP BY

```
String hql = "SELECT marks, COUNT(*) FROM Student GROUP BY marks";
```

8. Named Parameters

```
String hql = "FROM Student WHERE name = :name";

Query<Student> q = session.createQuery(hql, Student.class);
```

```
q.setParameter("name", "Indra");
```

9. LIKE

```
String hql = "FROM Student WHERE name LIKE 'I%'";
```

10. BETWEEN

```
String hql = "FROM Student WHERE marks BETWEEN 70 AND 90";
```

11. IN

```
String hql = "FROM Student WHERE id IN (101,102,103)";
```

Transaction Operations

```
Transaction tx = session.beginTransaction();

tx.commit();

tx.rollback();
```

Most Important CRUD Syntax for Exam

```
// CREATE
session.save(student);

// READ
session.createQuery("FROM Student", Student.class).list();

// UPDATE
session.createQuery(
"UPDATE Student SET marks=90 WHERE id=101"
).executeUpdate();

// DELETE
session.createQuery(
"DELETE FROM student WHERE id=101"
).executeUpdate();

// COMMIT
tx.commit();

// ROLLBACK
tx.rollback();
```

SQL vs HQL

SQL	HQL
SELECT * FROM student	FROM Student

SQL	HQL
UPDATE student SET marks=90	UPDATE Student SET marks=90
DELETE FROM student	DELETE FROM Student
Uses Table Names	Uses Java Class Names
Uses Column Names	Uses Object Properties

Remember: SQL → Table/Column names. HQL → Java Class/Object names. This is the biggest difference and a very common exam question.